

TP02 - Procesos ETL

Gustavo An Contardi, legajo 182818 Bases de Datos Masivas (11088) - 2do cuatrimestre 2026

Qué hice

Resolví las tres consignas dos veces sobre el mismo dataset, el Education Data del World Bank (43.624 filas, 69 columnas, formato ancho). Una vez con pandas en una notebook y otra con Apache Hop en el entorno dockerizado de la materia, con un pipeline por salida.

Salida	Filas	Pipeline de Hop
países.csv	266	tp02-A-paises.hpl
preguntas.csv	151	tp02-B-preguntas.hpl
primaria_2023.csv	256	tp02-C-primaria.hpl

Los tres archivos que genera Hop son idénticos a los que genera la notebook. Lo verifiqué con una función que compara por contenido y no por formato, porque las dos herramientas escriben distinto en cuestiones de comillas y saltos de línea.

Para que los archivos coincidan hubo que tomar una decisión que no es obvia: **ordenar explícitamente antes de numerar**. El id autoincremental depende del orden en que llegan las filas, así que si las dos implementaciones no ordenan igual, el mismo país queda con id distinto en cada archivo y la comparación falla aunque los datos estén bien. Lo mismo pasa con el desempate de la salida 3: 167 de los 256 países tienen 6 años de primaria, sin un segundo criterio de orden el resultado es arbitrario.

Estrategia para los nulos de la salida 3

De los 266 países del indicador SE.PRM.DURS, 255 reportan valor en 2023 y 11 no.

Adopté un criterio único: **arrastrar el último valor reportado, y descartar solo lo que no tiene ningún dato en los 64 años del archivo**. De los 11 sin dato en 2023, uno (Afganistán) tiene 53 años de historia y su último valor es de 2022, así que hereda ese 6. Los otros 10 nunca reportaron nada y quedan afuera. Resultado: 256 filas.

Elegí el criterio por ausencia de dato y no por tipo de entidad, que era la otra opción. Es reproducible y no depende de mi juicio sobre qué cuenta como país. Los 10 descartados quedan listados en C-descartados.csv, que genera el propio pipeline.

En pandas eso es un `ffill(axis=1)` sobre las columnas de año. En Hop es un transform `*Coalesce fields*`, que devuelve el primer valor no nulo de una lista de campos: pasándole los años de 2023 hacia atrás, el primero no nulo es el último reportado.

1. Facilidad de desarrollo y curva de aprendizaje

pandas me resultó bastante más intuitivo para las transformaciones de texto.

El patrón de la expresión regular es el mismo en las dos herramientas y es lo único realmente difícil del punto 2. Usé `\s*\([^()]*\)` en lugar de `\s*\(.*\)`, porque el cuantificador codicioso `matchea` desde el primer paréntesis hasta el último y rompe cinco indicadores del tipo `School enrollment`, `primary (gross)`, `gender parity index (GPI)`, donde se lleva puesto el `gender parity index`, que no estaba entre paréntesis.

La diferencia está en el envoltorio. En la notebook es una línea que se lee entera de un vistazo, con el patrón a la vista al lado de la operación. En Hop hay que llenar una grilla de nueve columnas (*In stream field*, *Out stream field*, *use RegEx*, *Search*, *Replace with*, y cuatro más), y el patrón queda escondido adentro de una celda: mirando el lienzo se ve un transform que dice "Sacar parentesis" y nada más. Para saber qué hace hay que abrirlo.

Sobre el tiempo, la curva fue muy marcada:

Salida	Tiempo en Hop
A (países)	~60 minutos
B (preguntas)	~20 minutos
C (primaria)	~20 minutos

Era mi primera vez con la herramienta, así que la primera hora fue casi toda aprendizaje y no complejidad del problema. Lo que me costó encontrar: que un transform se agrega con un clic en el lienzo y se configura con clic derecho → Edit; que la grilla de campos del *CSV file input* no se puede usar en la versión web porque queda con altura cero, y hay que ir a *Text file input*, que tiene el mismo trabajo repartido en pestañas; el editor de condición del *Filter rows*, donde la parte derecha son dos casillas apiladas y hay que saber que la de abajo es la del valor constante; y que los hops entre transforms hay que crearlos uno por uno a mano.

Nada de eso reaparece en el segundo pipeline. La tercera salida, que conceptualmente era la más difícil, me llevó el mismo tiempo que la segunda.

2. Manejo de errores, trazabilidad y depuración

Las dos herramientas te dejan mirar los datos del medio, pero de formas distintas.

Lo que Hop da gratis es el **conteo de filas por paso**. Cuando termina una ejecución, la pestaña Metrics muestra cuántas filas leyó y escribió cada transform. En la salida 3 eso se lee solo: 43.624 entran, 266 quedan después de filtrar el indicador, 256 pasan el filtro de nulos y 10 se van por la otra rama. En la notebook esa misma trazabilidad existe, pero la tengo que escribir yo con prints en cada paso.

Lo segundo que da Hop es el patrón de rechazos: un *Filter rows* tiene dos salidas, y lo que no pasa el filtro se puede escribir a un archivo en vez de desaparecer. Los 10 países descartados de la salida 3 no son un número en un log, son un CSV que puedo abrir.

En contra, tres cosas concretas que me pasaron:

- El preview no funciona igual en todos los transforms. Sobre un *Filter rows* devuelve "no preview rows found" aunque el pipeline corra bien, porque las filas salen por las ramas con nombre y no por una salida común. Hay que mirar un transform después, o ir a Metrics.
- Los nombres de campo se validan recién en ejecución. Escribí `duracion_primaria_datos` en lugar de `duracion_primaria_anios` en el *Sort rows* y Hop me dejó guardar sin chistar; el error saltó cuando pasó la primera fila.
- No todos los errores rojos son del pipeline. Al guardar por primera vez me tiró `SWTException: Invalid thread access` con un stack trace largo, y el archivo se había guardado igual. En otra corrida un problema de permisos del entorno se presentó como `preparing pipeline execution failed` con veinte líneas de traza que no tenían nada que ver con lo que yo había armado.

En la notebook los errores apuntan a la línea y puedo inspeccionar cualquier variable intermedia con una celda nueva, sin volver a correr nada. La contra es el estado: las celdas se pueden ejecutar en cualquier orden, así que una variable puede tener un valor que no corresponde al código que estoy leyendo. En Hop eso no puede pasar porque el pipeline arranca siempre desde el archivo.

Para explorar y entender los datos me quedo con la notebook. Para una corrida que después hay que auditar, los contadores por paso y los archivos de rechazo de Hop son más difíciles de falsear.

3. Escalabilidad y rendimiento en procesamiento masivo

La limitación de pandas es de modelo, no de tamaño: `read_csv` levanta el archivo entero en memoria y el `DataFrame` vive ahí. Mientras entre, anda rápido; cuando no entra, no hay un modo degradado, directamente falla. Las salidas son `chunks`, Dask o Polars, y las tres implican reescribir el código, no cambiarle un parámetro.

Hop trabaja por filas. Los transforms de un pipeline arrancan todos a la vez y las filas los van atravesando de a una, así que la memoria que se usa depende del tamaño de los buffers entre transforms y no del tamaño del archivo. Leer un CSV, filtrar y escribir otro se puede hacer sobre un archivo más grande que la RAM sin cambiar nada.

La excepción son los transforms que bloquean, que necesitan ver todo antes de emitir. El `*Sort rows*` es el caso claro, y es interesante porque Hop lo resuelve a la vista: en su diálogo hay un campo `*Sort size (rows in memory)*` y un directorio temporal. Cuando la entrada supera ese tope, el transform escribe archivos intermedios a disco y los mezcla. O sea que ordenar no está limitado por la RAM, está limitado por el disco y cuesta más lento. Lo mismo aplica al `*Unique rows*`, que exige la entrada ordenada.

Para escalar más allá de una máquina, Hop permite ejecutar el mismo pipeline sobre Apache Beam cambiando la configuración de ejecución, con Spark, Flink o Dataflow como motor. En la instalación que usé aparecen los transforms de Beam en el catálogo. No es gratis: el conjunto de transforms soportados sobre Beam es más acotado que el local, así que no cualquier pipeline se migra sin tocar nada. Pero la unidad de trabajo sigue siendo el mismo archivo `.hpl`, no un código distinto.

En este TP el volumen no fue un problema para ninguna de las dos. El contenedor de Hop está limitado a 1 GB de heap y procesó las 43.624 filas sin despeinarse.

4. Portabilidad, automatización y despliegue en producción

La ventaja más concreta de dockerizar Hop es que el entorno deja de ser un problema. Un `docker-compose up` levanta la versión 2.19.0 exacta, con sus plugins y su JDK, sin instalar nada en mi máquina. La carpeta del proyecto está montada como volumen, así que los pipelines que edito desde el navegador son archivos del repositorio.

Y son archivos de texto. Un `.hpl` es XML: se versiona, se puede revisar en un pull request y se ve qué cambió entre dos versiones. Un `.ipynb` es JSON con las salidas adentro, y el diff de una notebook es prácticamente ilegible.

Para automatizar, Hop trae `hop-run`, que ejecuta un pipeline desde la línea de comandos sin interfaz gráfica. Eso es lo que permite meterlo en un cron o en un contenedor efímero que arranca, corre y termina. La GUI queda como herramienta de diseño, no como requisito de ejecución. Con notebooks hay que sumar `papermill` o `nbconvert`, y además fijar el entorno de Python, que es justamente lo que Docker resuelve del otro lado.

Hop también tiene workflows (`.hwf`), que encadenan pipelines en secuencia y ramifican según el resultado de cada paso: si uno falla, el flujo se va por la rama de error en vez de seguir con datos

incompletos.

Dicho eso, no son herramientas equivalentes y no las pondría a competir. Un workflow de Hop orquesta lo que pasa adentro de Hop. Airflow orquesta un sistema: calendarios, reintentos, dependencias entre procesos de distintas herramientas, backfills, alertas. Lo razonable es combinarlos, con Airflow disparando un hop-run dentro de un contenedor, y ahí la dockerización vuelve a ser la ventaja, porque el contenedor es la unidad que Airflow sabe manejar.

Limitaciones asumidas

- El dataset incluye agregados regionales (world, Euro area, High income) además de países. Los dejé adentro de las tres salidas, porque la consigna pide "país o entidad regional". world aparece en el ranking de primaria con 6 años.
- Al sacar el contenido entre paréntesis, 26 indicadores distintos colapsan en 13 nombres repetidos. Por ejemplo School enrollment, primary (% gross) y (% net) quedan los dos como School enrollment, primary. Los consolidé porque la consigna pide indicadores únicos, pero eso significa que preguntas.csv pierde la distinción entre indicadores que miden cosas diferentes.